

Day 34 : Promise in JS

The **Promise** object represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

First Principle: A Promise is a Placeholder

Promise Characteristics:

1. **A Promise is an object** - You can store it in a variable
2. **Represents future value** - The value isn't available yet, but will be
3. **Has states** - It changes state over time
4. **One-time use** - Once settled, it never changes

Promise States (First Principles)

A Promise can be in **exactly ONE** of three states at any time:



1. PENDING (Initial state)

↓

"I'm working on it..."

↓

├── Success → 2. FULFILLED (Resolved)

│ "I got the result!"

└── Failure → 3. REJECTED

│ "Something went wrong!"

Once **FULFILLED** or **REJECTED**, the promise is **SETTLED** (final)

Visual Example:

```
// Create a promise
const myPromise = new Promise((resolve, reject) => {
  // At this moment: Promise is PENDING

  setTimeout(() => {
    const success = true;

    if (success) {
      resolve("Success!"); // → Promise becomes FULFILLED
    } else {
      reject("Failed!"); // → Promise becomes REJECTED
    }
  }, 2000);
});

// myPromise is PENDING for 2 seconds
// Then becomes FULFILLED with value "Success!"
```

State Transitions:

```
const promise = new Promise((resolve, reject) => {
  console.log("State: PENDING");

  setTimeout(() => {
    resolve("Done");
    console.log("State: FULFILLED");

    // ❌ This does nothing - promise is already settled!
    reject("Error"); // Ignored!
    resolve("Again"); // Ignored!
  });
});
```

```
    }, 1000);  
  });
```

// Key Point: A promise can only settle ONCE

Creating Promises

Basic Syntax:

```
const promise = new Promise((resolve, reject) => {  
  // executor function  
  // This runs IMMEDIATELY when promise is created  
});
```

The Executor Function:

```
new Promise((resolve, reject) => {  
  // Two parameters:  
  // - resolve: function to call when operation succeeds  
  // - reject: function to call when operation fails  
  
  // Your async operation here  
});
```

Example 1: Simple Promise

```
const simplePromise = new Promise((resolve, reject) => {  
  resolve("Hello!");  
});  
  
// This promise immediately resolves with "Hello!"
```

Example 2: Promise with setTimeout

```
const delayedPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Done after 2 seconds");
  }, 2000);
});
```

Example 3: Promise with Conditional Logic

```
function checkAge(age) {
  return new Promise((resolve, reject) => {
    if (age >= 18) {
      resolve("Access granted");
    } else {
      reject("Access denied - Too young");
    }
  });
}

// Usage:
const ageCheck = checkAge(20); // Returns a promise
```

Example 4: Real-World - Simulating API Call

```
function fetchUserData(userId) {
  return new Promise((resolve, reject) => {
    console.log(`Fetching user ${userId}...`);

    setTimeout(() => {
      // Simulate successful API response
      const user = {
        id: userId,
        name: "John Doe",
        email: "john@example.com"
      };
    });
  });
}
```

```
    resolve(user);  
  }, 2000);  
});  
}
```

Example 5: Promise with Error Handling

```
function divideNumbers(a, b) {  
  return new Promise((resolve, reject) => {  
    if (b === 0) {  
      reject("Cannot divide by zero!");  
    } else {  
      resolve(a / b);  
    }  
  });  
}
```

Consuming Promises

Once you have a promise, you need to **consume** it to get the result.

Method 1: Using .then()

```
promise.then((result) => {  
  // This function runs when promise is FULFILLED  
  console.log(result);  
});
```

Method 2: Using .catch()

```
promise.catch((error) => {  
  // This function runs when promise is REJECTED
```

```
console.error(error);  
});
```

Method 3: Using .finally()

```
promise.finally(() => {  
  // This runs ALWAYS, whether fulfilled or rejected  
  console.log("Promise settled");  
});
```

Complete Example:

```
function fetchData() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      const success = Math.random() > 0.5;  
  
      if (success) {  
        resolve({ data: "Some data" });  
      } else {  
        reject("Network error");  
      }  
    }, 1000);  
  });  
}
```

// Consuming the promise:

```
fetchData()  
  .then((result) => {  
    console.log("Success:", result);  
  })  
  .catch((error) => {  
    console.error("Error:", error);  
  })  
  .finally(() => {
```

```
console.log("Request completed");
});
```

Understanding .then() Parameters:

`.then()` can actually take TWO functions:

```
promise.then(
  (result) => {
    // Success handler
    console.log("Success:", result);
  },
  (error) => {
    // Error handler (optional)
    console.error("Error:", error);
  }
);

// But it's better to use .catch() for errors:
promise
  .then((result) => {
    console.log("Success:", result);
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

Promise Chaining (The Power of Promises!)

This is where Promises truly shine - **flat, sequential async operations**.

First Principle: .then() Returns a New Promise

```
promise
  .then((result) => {
```

```

    // Whatever you return here becomes the value
    // of the NEXT .then()
    return result * 2;
  })
  .then((doubledResult) => {
    console.log(doubledResult);
  });

```

Example 1: Basic Chaining

```

Promise.resolve(5)
  .then((value) => {
    console.log(value); // 5
    return value * 2;
  })
  .then((value) => {
    console.log(value); // 10
    return value + 3;
  })
  .then((value) => {
    console.log(value); // 13
  });

```

Example 2: Chaining Async Operations

```

function step1() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 1 complete");
      resolve("Result from step 1");
    }, 1000);
  });
}

function step2(previousResult) {

```



```

return new Promise((resolve) => {
  setTimeout(() => {
    console.log("Step 2 complete, got:", previousResult);
    resolve("Result from step 2");
  }, 1000);
});
}

```

```

function step3(previousResult) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 3 complete, got:", previousResult);
      resolve("Final result");
    }, 1000);
  });
}

```

//  Clean chain (no nesting!)

```

step1()
  .then((result1) => {
    return step2(result1);
  })
  .then((result2) => {
    return step3(result2);
  })
  .then((finalResult) => {
    console.log("All done:", finalResult);
  })
  .catch((error) => {
    console.error("Something failed:", error);
  });

```

// Even cleaner (shorthand):

```

step1()
  .then(step2)
  .then(step3)

```

```

.then((finalResult) => {
  console.log("All done:", finalResult);
})
.catch((error) => {
  console.error("Something failed:", error);
});

```

The .finally() Method

`.finally()` runs **regardless** of whether the promise succeeds or fails.

When to Use .finally()

Use `.finally()` for **cleanup code** that must always run:

- Hide loading spinners
- Close database connections
- Re-enable buttons
- Stop timers
- Release resources

Example 1: Loading Spinner

```

function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = Math.random() > 0.5;
      if (success) {
        resolve({ data: "Data loaded" });
      } else {
        reject("Failed to load");
      }
    }, 2000);
  });
}

```

```

// Show spinner
console.log("🔄 Loading...");
let isLoading = true;

fetchData()
  .then((result) => {
    console.log("✅ Success:", result);
  })
  .catch((error) => {
    console.error("❌ Error:", error);
  })
  .finally(() => {
    // Always runs - hide spinner
    isLoading = false;
    console.log("🛑 Loading stopped");
  });

```

Example 2: Database Connection

```

function queryDatabase(query) {
  let connection;

  return connectToDatabase()
    .then((conn) => {
      connection = conn;
      return connection.execute(query);
    })
    .then((results) => {
      console.log("Query results:", results);
      return results;
    })
    .catch((error) => {
      console.error("Query failed:", error);
      throw error;
    });
}

```

```

    })
    .finally(() => {
      // Always close connection
      if (connection) {
        connection.close();
        console.log("Connection closed");
      }
    });
  }
}

```

Key Characteristics of .finally()

1. Doesn't receive any arguments

```

promise
  .then((result) => {
    console.log("Result:", result); // Has result
  })
  .catch((error) => {
    console.log("Error:", error); // Has error
  })
  .finally(() => {
    // No arguments!
    console.log("Done");
  });

```

1. Doesn't change the promise value

```

Promise.resolve("Original")
  .finally(() => {
    return "Modified"; // Ignored!
  })
  .then((value) => {
    console.log(value); // Still "Original"
  });

```

1. But if `.finally()` throws, that error propagates

```
Promise.resolve("Success")
  .finally(() => {
    throw new Error("Cleanup failed");
  })
  .then((value) => {
    console.log(value); // SKIPPED
  })
  .catch((error) => {
    console.error(error.message); // "Cleanup failed"
  });
```

JSON vs JavaScript Object

Quick Visual Comparison

```
// JavaScript Object
const jsObject = {
  name: "John",      // No quotes around key
  age: 30,           // Number
  active: true,       // Boolean
  greet: function() { // Can have functions
    console.log("Hi!");
  },
  date: new Date(),   // Can have Date objects
  undefined: undefined, // Can have undefined
};

// JSON (String format)
const jsonString = `{
```

```

    "name": "John",
    "age": 30,
    "active": true
  };
  // Note: Must use double quotes, no functions, no undefined, etc.

```

Key Differences

Feature	JavaScript Object	JSON
Type	JavaScript data structure	Text/String format
Keys	Can be unquoted	Must use double quotes "key"
Strings	Single ' or double " quotes	Only double quotes "
Data Types	String, Number, Boolean, Array, Object, Function, Date, undefined, null, Symbol, etc.	String, Number, Boolean, Array, Object, null only
Functions	✅ Allowed	❌ Not allowed
undefined	✅ Allowed	❌ Not allowed
Date objects	✅ Allowed	❌ Not allowed (stored as string)
Comments	✅ Allowed	❌ Not allowed
Trailing commas	✅ Allowed	❌ Not allowed

Detailed Examples

1. Key Quotes

```

// JavaScript Object - keys can be unquoted
const jsObj = {
  name: "John",
  age: 30
};

```

```
// JSON - keys MUST be double-quoted
const json = `{
  "name": "John",
  "age": 30
}`;
```

2. String Quotes

```
// JavaScript Object - single or double quotes
const jsObj = {
  name: 'John',    // Single quotes OK
  city: "New York" // Double quotes OK
};
```

```
// JSON - ONLY double quotes
const json = `{
  "name": "John",
  "city": "New York"
}`;
```

3. Functions

```
// JavaScript Object - can have functions
const jsObj = {
  name: "John",
  greet: function() {
    return "Hello!";
  }
};
```

```
// JSON - NO functions allowed
const json = `{
  "name": "John"
}`;
```

```
};  
// "greet" would cause an error in JSON
```

4. undefined vs null

```
// JavaScript Object - can have undefined  
const jsObj = {  
  name: "John",  
  age: undefined, // OK  
  city: null      // OK  
};  
  
// JSON - NO undefined, but null is OK  
const json = `{  
  "name": "John",  
  "city": null  
}`;  
// "age": undefined would be invalid JSON
```

5. Trailing Commas

```
// JavaScript Object - trailing commas OK  
const jsObj = {  
  name: "John",  
  age: 30, // ← Trailing comma OK  
};  
  
// JSON - NO trailing commas  
const json = `{  
  "name": "John",  
  "age": 30  
}`;  
// Adding a comma after 30 would be invalid JSON
```


6. Comments

```
// JavaScript Object - comments allowed
const jsObj = {
  name: "John", // This is a comment
  age: 30
};

// JSON - NO comments allowed
const json = `{
  "name": "John",
  "age": 30
}`;
```

Converting Between JSON and JavaScript Objects

JavaScript Object → JSON String

```
const user = {
  name: "John",
  age: 30,
  active: true
};

// Convert to JSON string
const jsonString = JSON.stringify(user);
console.log(jsonString);
// Output: {"name":"John","age":30,"active":true}

console.log(typeof jsonString); // "string"
```

JSON String → JavaScript Object

```
const jsonString = '{"name":"John","age":30,"active":true}';

// Convert to JavaScript object
const user = JSON.parse(jsonString);
console.log(user);
// Output: { name: 'John', age: 30, active: true }

console.log(typeof user); // "object"
console.log(user.name); // "John"
```

Why Does `fetch()` Return JSON?

```
fetch("https://api.github.com/users")
  .then(response => {
    // response.body is a stream of JSON text
    console.log(typeof response); // "object" (Response object)

    return response.json(); // Parses JSON string → JS object
  })
  .then(data => {
    // Now 'data' is a JavaScript object
    console.log(typeof data); // "object" (JavaScript object)
    console.log(data[0].login); // Can access properties
  });
```

The flow:

1. Server sends data as **JSON string** (text)
2. `fetch()` receives it as a **Response object**
3. `.json()` parses the JSON string → **JavaScript object**
4. Now you can use it like a normal JS object

Practical Example

```
// Creating a user object in JavaScript
const user = {
  name: "Alice",
  age: 25,
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
};

user.greet(); // Works fine

// Sending to server (must convert to JSON)
const jsonToSend = JSON.stringify(user);
console.log(jsonToSend);
// Output: {"name":"Alice","age":25}
// Notice: greet() function is gone! JSON can't store functions

// Receiving from server
const jsonReceived = '{"name":"Bob","age":30}';
const userFromServer = JSON.parse(jsonReceived);
console.log(userFromServer.name); // "Bob"
// userFromServer.greet(); // ❌ Error - no function exists
```

Summary

- **JavaScript Object:** Native JS data structure, can hold functions, dates, undefined, etc.
- **JSON:** Text format for data exchange, strict rules, only basic data types
- **JSON.stringify():** Convert JS object → JSON string
- **JSON.parse():** Convert JSON string → JS object
- **fetch().json():** Automatically parses JSON response → JS object

Think of JSON as the "language" different systems use to talk to each other, while JavaScript objects are what you actually work with in your code!

